Array Problem Solving Patterns (Java)

1. Two Pointer Technique

When to Use

Use two pointers when:

- Array is sorted
- Need pairs/triplets
- Need partitioning
- Need opposite-end traversal
- Need in-place modifications

Recognition Keywords

- Pair with target sum
- Triplet sum
- Sorted array
- Remove duplicates
- Container problems
- Closest pair

Pattern 1: Opposite Direction Pointers

Example

Find two numbers whose sum equals target.

```
int left = 0;
int right = arr.length - 1;

while(left < right) {

    int sum = arr[left] + arr[right];
```

```
        if(sum == target) {
            return new int[]{left, right};
        }

        if(sum < target) {
            left++;
        } else {
            right--;
        }
    }
```

Complexity

```
Time : O(n)
Space: O(1)
```

Pattern 2: Same Direction Pointers

Example

Remove duplicates from sorted array.

```
int slow = 0;

for(int fast = 1; fast < nums.length; fast++) {

    if(nums[fast] != nums[slow]) {
        slow++;
        nums[slow] = nums[fast];
    }
}

return slow + 1;
```

Generic Java Template

```
class Solution {

    public int solve(int[] nums) {

        int left = 0;
        int right = nums.length - 1;

        while(left < right) {

            // process

            left++;
            right--;
        }

        return 0;
    }
}
```

---

2. Sliding Window

Fixed Size Window

Recognition Keywords

- Subarray of size K
- Average of K elements
- Maximum sum K
- Window size fixed

Generic Template

```
class Solution {

    public int solve(int[] nums, int k) {
```

```
        int windowSum = 0;

        for(int i=0;i<k;i++) {
            windowSum += nums[i];
        }

        int answer = windowSum;

        for(int i=k;i<nums.length;i++) {

            windowSum += nums[i];
            windowSum -= nums[i-k];

            answer = Math.max(answer, windowSum);
        }

        return answer;
    }
}
```

Important Problems

LeetCode

- Maximum Average Subarray I
- Sliding Window Maximum
- Grumpy Bookstore Owner

HackerRank

- Max Sum Subarray of Size K

Variable Size Window

Recognition Keywords

- Longest

- Shortest
- At most K
- At least K
- Distinct characters
- Positive numbers

Generic Template

```
class Solution {

    public int solve(int[] nums, int k) {

        int left = 0;
        int answer = 0;

        int current = 0;

        for(int right=0; right<nums.length; right++) {

            current += nums[right];

            while(current > k) {
                current -= nums[left++];
            }

            answer = Math.max(answer, right - left + 1);
        }

        return answer;
    }
}
```

Common Problems

LeetCode

- Longest Substring Without Repeating Characters
- Minimum Size Subarray Sum
- Fruit Into Baskets

- Max Consecutive Ones III

GFG

- Smallest Subarray with Sum Greater Than X

---

3. Prefix Sum

Recognition Keywords

- Range Sum
- Subarray Sum
- Query
- Count subarrays

Concept

```
prefix[i]
=
sum of elements from 0 to i
```

Build Prefix Array

```
int[] prefix = new int[n];

prefix[0] = nums[0];

for(int i=1;i<n;i++) {
    prefix[i] = prefix[i-1] + nums[i];
}
```

Range Sum Query

```
sum(l,r)

=
prefix[r] - prefix[l-1]
```

Prefix Sum Template

```
class Solution {

    public int[] buildPrefix(int[] nums) {

        int n = nums.length;

        int[] prefix = new int[n];

        prefix[0] = nums[0];

        for(int i=1;i<n;i++) {
            prefix[i] = prefix[i-1] + nums[i];
        }

        return prefix;
    }
}
```

Prefix Sum + HashMap Pattern

Subarray Sum Equals K

```
Map<Integer,Integer> map = new HashMap<>();

map.put(0,1);

int prefix = 0;
int count = 0;
```

```
for(int num : nums) {

    prefix += num;

    count += map.getOrDefault(prefix-k,0);

    map.put(prefix,
            map.getOrDefault(prefix,0)+1);
}
```

Important Problems

LeetCode

- Subarray Sum Equals K
- Continuous Subarray Sum
- Range Sum Query

GFG

- Zero Sum Subarrays

---

4. Dual Array Operations

Merge Two Sorted Arrays

Recognition

```
Two sorted arrays
Merge output sorted
```

Template

```
class Solution {

    public int[] merge(int[] a, int[] b) {

        int i = 0;
        int j = 0;

        List<Integer> result = new ArrayList<>();

        while(i < a.length && j < b.length) {

            if(a[i] <= b[j]) {
                result.add(a[i++]);
            } else {
                result.add(b[j++]);
            }
        }

        while(i < a.length)
            result.add(a[i++]);

        while(j < b.length)
            result.add(b[j++]);

        return result.stream()
                .mapToInt(Integer::intValue)
                .toArray();
    }
}
```

Union of Sorted Arrays

```
while(i < n && j < m) {

    if(a[i] < b[j])
        add(a[i++]);
```

```
    else if(a[i] > b[j])
        add(b[j++]);

    else {
        add(a[i]);
        i++;
        j++;
    }
}
```

Intersection of Sorted Arrays

```
while(i < n && j < m) {

    if(a[i] < b[j]) {
        i++;
    }
    else if(a[i] > b[j]) {
        j++;
    }
    else {
        result.add(a[i]);
        i++;
        j++;
    }
}
```

---

5. Dutch National Flag Algorithm

Recognition

```
Sort array containing:
0 1 2

or
```

3 categories

Example

```
int low = 0;
int mid = 0;
int high = nums.length - 1;

while(mid <= high) {

    if(nums[mid] == 0) {

        swap(nums, low, mid);
        low++;
        mid++;
    }
    else if(nums[mid] == 1) {

        mid++;
    }
    else {

        swap(nums, mid, high);
        high--;
    }
}
```

Complexity

```
Time : O(n)
Space: O(1)
```

Problems

LeetCode

- Sort Colors (#75)

---

6. Cyclic Sort

Recognition Keywords

```
Numbers 1 to n
Missing number
Duplicate number
Corrupt pair
```

Core Idea

Put each element in its correct position.

```
value x

should be at

index x - 1
```

Generic Template

```
int i = 0;

while(i < nums.length) {

    int correct = nums[i] - 1;

    if(nums[i] != nums[correct]) {

        swap(nums, i, correct);
    }
    else {
```

```
            i++;
        }
    }
```

Problems

LeetCode

- Missing Number
- Find Duplicate Number
- First Missing Positive
- Set Mismatch

Hands-On Section

A. Subarray & Window Problems

Easy

1. Maximum Sum Subarray Size K
2. Average of Size K
3. Longest Ones

Medium

4. Longest Substring Without Repeating Characters
5. Fruit Into Baskets
6. Minimum Size Subarray Sum

Hard

7. Sliding Window Maximum
8. Minimum Window Substring

B. Maximum / Minimum Pattern Problems

Easy

1. Maximum Subarray
2. Best Time To Buy And Sell Stock

Medium

3. Maximum Product Subarray
4. Maximum Circular Subarray

Hard

5. Maximum Sum Rectangle

(Kadane PDF covers these extensively.)

C. Product Based Problems

Maximum Product Subarray

Key Idea

Need both min and max because:

```
negative × negative
=
positive
```

Template

```
class Solution {
```

```
    public int maxProduct(int[] nums) {

        int max = nums[0];
        int min = nums[0];
        int answer = nums[0];

        for(int i=1;i<nums.length;i++) {

            if(nums[i] < 0) {

                int temp = max;
                max = min;
                min = temp;
            }

            max = Math.max(nums[i],
                           max * nums[i]);

            min = Math.min(nums[i],
                           min * nums[i]);

            answer = Math.max(answer, max);
        }

        return answer;
    }
}
```

Complete Competitive Programming Array Template

```
import java.util.*;

class Solution {

    private void swap(int[] arr,
                      int i,
                      int j) {
```

```
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    private int[] buildPrefix(int[] nums) {

        int[] prefix = new int[nums.length];

        prefix[0] = nums[0];

        for(int i=1;i<nums.length;i++) {
            prefix[i] = prefix[i-1] + nums[i];
        }

        return prefix;
    }

    public int solve(int[] nums) {

        // implement logic

        return 0;
    }
}
```

Recommended Practice Order

1. Two Sum II
2. Remove Duplicates from Sorted Array
3. Maximum Sum Subarray of Size K
4. Longest Substring Without Repeating Characters
5. Subarray Sum Equals K
6. Merge Sorted Array
7. Union of Two Sorted Arrays (GFG)
8. Intersection of Two Arrays II
9. Sort Colors
10. Missing Number
11. Find All Duplicates in an Array
12. Maximum Product Subarray
13. Maximum Circular Subarray
14. Sliding Window Maximum

15. Minimum Window Substring

This progression mirrors the pattern-learning path typically followed in LeetCode, HackerRank, and GeeksforGeeks interview preparation tracks.